COMP 421 - Database Systems

Assignment 2: Query Evaluation

Yazdan Zinati
yazdan.zinati@mail.mcgill.ca
260928077

March 24, 2022

Ex. 1

a
First we find tuple sizes for each relation.
Buisness:
`bid (CHAR(10))`: $10 \times 1\,byte = 10\,bytes$
`btype(VARCHAR(30))`: $^2/_3 \times 30\,bytes = 20\,bytes$
`bname(VARCHAR(60))`: $^2/_3 \times 60\,bytes = 40\,bytes$
`bcity(VARCHAR(30))`: $^2/_3 \times 30\,bytes = 20\,bytes$
`baddr(VARCHAR(60))`: $^2/_3 \times 60\,bytes = 40\,bytes$
`province(CHAR(2))`: $2 \times 1\,byte = 2\,bytes$
`bstartdate(DATE)`: $10 bytes$
lenght of a `Buisness` tuple = 10 + 20 + 40 + 20 + 40 + 2 + 10 = 142 bytes
each page has a capacity of $0.75 \times 4000 = 3000\,bytes$
total size of the `Buisness` relation = $CARD(Buisness) \times tuplesize = 20000 \times 142\,bytes$

Thus we require the following number of pages: $\frac{20000 \times 142}{3000} = 947\,pages$

Users:
`uemail(VARCHAR(45))`: $^2/_3 \times 45\,bytes = 30\,bytes$
`uname(VARCHAR(45))`: $^2/_3 \times 45\,bytes = 30\,bytes$
`udateofbirth(DATE)`: $10\,bytes$
`ujoindate(DATE)`: $10\,bytes$
`uprovince(CHAR(2))`: $2 \times 1\,bytes = 2\,bytes$
lenght of a `Users` tuple = 30 + 30 + 10 + 10 + 2 = 82 bytes
total size of the `Users` relation = $CARD(Users) \times tuplesize = 90000 \times 82\,bytes$

Thus we require the following number of pages: $\frac{90000 \times 82}{3000} = 2460\,pages$

Reviews:
`reviewud(INT)`: $8\,bytes$
`bid(CHAR(10))`: $10 \times 1\,byte = 10\,bytes$
`uemail(VARCHAR(45))`: $^2/_3 \times 45\,bytes = 30\,bytes$
`rstars(SHORT)`: $1\,byte$
`rtext(VARCHAR(600))`: $^2/_3 \times 600\,bytes = 400\,bytes$
`rdate(DATE)`: $10\,bytes$
lenght of a `Review` tuple = 8 + 10 + 30 + 1 + 400 + 10 = 459 bytes
CARD(Reviews) = avg reviews per user × CARD(Users) = 20 × 90000

Thus we require the following number of pages: $\frac{90000 \times 20 \times 459}{3000} = 275400\,pages$

b
(i)
Assuming a uniform distribution of reviews per buisnesses:
number of qualifying records = $\frac{CARD(Reviews)}{CARD(Buisness)} = \frac{CARD(Users) \times avgreviewsperuser}{CARD(Buisness)} = \frac{90000 \times 20}{20000} = 90\,tuples$
Since there are 90 tuples, this could result in reading 90 pages (since index is unclusteres and each tuple could be in a different data page).
size of `bid` = 10 bytes and size of `rid` = 10 bytes so each leaf page can contain $\frac{0.75 \times 4000}{20} = 150\,entries$
Since $90 < 150$, only one leaf page has to be read.

Thus total cost = 1 leaf page + 90 data pages = 91 pages of IO cost.

(ii) Since the index in clustered only the first entry leaf page is read.
We found in part (a) that reviews has 275400 pages. Assuming a uniform distribution on reviews per buisnesses, we read $\frac{275400}{20000} = 14\,pages$.

Thus total cost = 1 leaf page + 14 data pages = 15 pages of IO cost.

(iii) Same as in previous parts, 90 tuples qualify only based on rid. However, we can use the rstars in the `WHERE` clause to reduce IO cost.
We know the distribution of rstars (15% for 5 stars, 25% for 4, 3, and 2 stars and 10% for 1 star). Therefore, if `N = 1`, 100% of the qualifying 90 records match. If `N = 2`, 90% of the 90 tuples (81) match. For `N = 3`, 65% of the 90 records match (58.5). For `N = 4`, 40% of the 90 records match (36). Finally, for `N = 5`, 15% of the 90 tuples (13.5) satisfy the condition.
Therefore, the cost will be different depending on the value on N.
Only 1 leaf page will be visited since all data entries fit in 3000 bytes. Since the data entries are grouped by both bid and rstars, the data entries will be less than $(10+1) \times 90 + 10 \times 90$ which itself fits in one page. However, not all 90 data pages are visited since less data pages are visited given the value of N.
The cost will depend on the reduction factor that depends on the value of N. If the qualifying tuples form the relation T then:

$$cost = 90 \times reductionfactor(|\sigma_{N>=rstars}(T)|/|T|) + 1$$

if N = 1: cost = 1×90 data pages + 1 leaf page = 91 IO
if N = 2: cost = 0.9×90 data pages + 1 leaf page = 82 IO
if N = 3: cost = 0.65×90 data pages + 1 leaf page = 60 IO
if N = 4: cost = 0.4×90 data pages + 1 leaf page = 37 IO
if N = 5: cost = 0.15×90 data pages + 1 leaf page = 15 IO

Ex. 3

(a)
To perform algebraic optimizayion, we first push down selection and projection operators and avoid cross products to limit the number of tuples flowing from one operator to the next.

We first perform optimization on each relation.
$\rho(tr, \Pi_{rstars,rdate,bid,uemail}(Reviews))$
$\rho(tu, \Pi_{uemail}(\sigma_{dateofbirth \geq '1990-01-01' \wedge dateofbirth \leq '1990-12-31'}(Users)))$
$\rho(tb, \Pi_{bname,bprovince,bid}(\sigma_{type='restaurant'}(Buisness)))$

Since we ignore cardinalities in this question, we have 2 solutions:

solution 1:
$\rho(Temp, \Pi_{bname,province,rstars,rdate,uemail}(tb \bowtie_{tb.bid=tr.bid} tr))$
$\Pi_{bname,province,uemail,rstars,rdate}(Temp \bowtie_{Temp.uemail=tu.uemail} tu)$

solution 2:
$\rho(Temp, \Pi_{tu.uemail,rstars,rdate,bid}(tr \bowtie_{tr.uemail=tu.uemail} tu))$
$\Pi_{bname,bprovince,uemail,rstars,rdate}(Temp \bowtie_{Temp.bid=tb.bid} tb)$

(b)
We chose Solution 1 found in part (a) for this answer. See Figure 1 for execution plan tree. We first start by looking at the best algorithm for joining the Review and Buisness relations. We don't consider simple nested and page nested join loop algorithms since they are not efficient compared to other algorithms. Since we have an index on `bid` of the Buisness table, lets start by finding the cost of an index nexted join loop. Since the index is on Buisness, Reviews will be outer and Buisness will be the inner relation.
cost = ReviewPages + CARD(Review) $\times$ cost of finding = $275400 + 20 \times 90000 \times 2$.
Cost of finding is 2 since bid is unique in Buisness (1 leaf + 1 data page).

This (Index nested join loop) is incredibly inneficient since the index is on a small relation compared to its outer relation (Reviews). We could have gained great IO if the index was on bid of Revies instead.

We now consider block nested join loop. We will get best results if the outer relation is small and has a selection. So Buisness is the outer relation and Reviews is the inner relation. Note that for simplicity of calculations, we are

assuming a few more buffer frames (allowed in instructions). Note: we pipeline qualifying Buisness tuples into the join

cost = Buisness pages + $\lceil \frac{BuisnessPages}{100 \times 100} \rceil \times$ Review Pages = $947 + \lceil 947 \rceil 10000 \times 275400 = 947 + 275400 = 276347$ IO (we have 100 buisness types uniformly distributed thus a reduction factor of 0.01)

This cost is minimum and can't be beaten by sort-merge or hash join since the cost is equal to: $Outerpages + Innerpages$.

We have defered the projection of attributed of Review and Buisness after the join. As we saw, this does not make any difference in terms of IO cost calculations. Furthermore, records fit into memory already.
We call the output of the Join, T. This intermediate relation will contain $\frac{20 \times 90000}{100} = 18000$ tuples.
The total size of the tuple in T will be 1 + 10 + 30 + 40 + 2 = 83 bytes
The size of T will be $83 \times 18000 = 1494000$ bytes. Relation T will fit in $\lceil {}^{1494000}/_{4000} \rceil = 374\, pages$.
Since it can't fit into memory, we will write it to disk (IO cost = 374).

Only 1825 users are born in 1990 and the record size after projection on `uemail` is 30×1825. This needs $\lceil \frac{30 \times 1825}{4000} \rceil = 14$ pages which we can keep in memory.

A block nested join loop with User as outer will give us good IO cost (Note: we pipeline qualifying user tuples into the join):
cost = User Pages + $\lceil 14/100 \rceil \times Tpages = 2460 + 374 = 2834$ IO.

The total cost will be 276347 + 374 + 2834 = 279555.

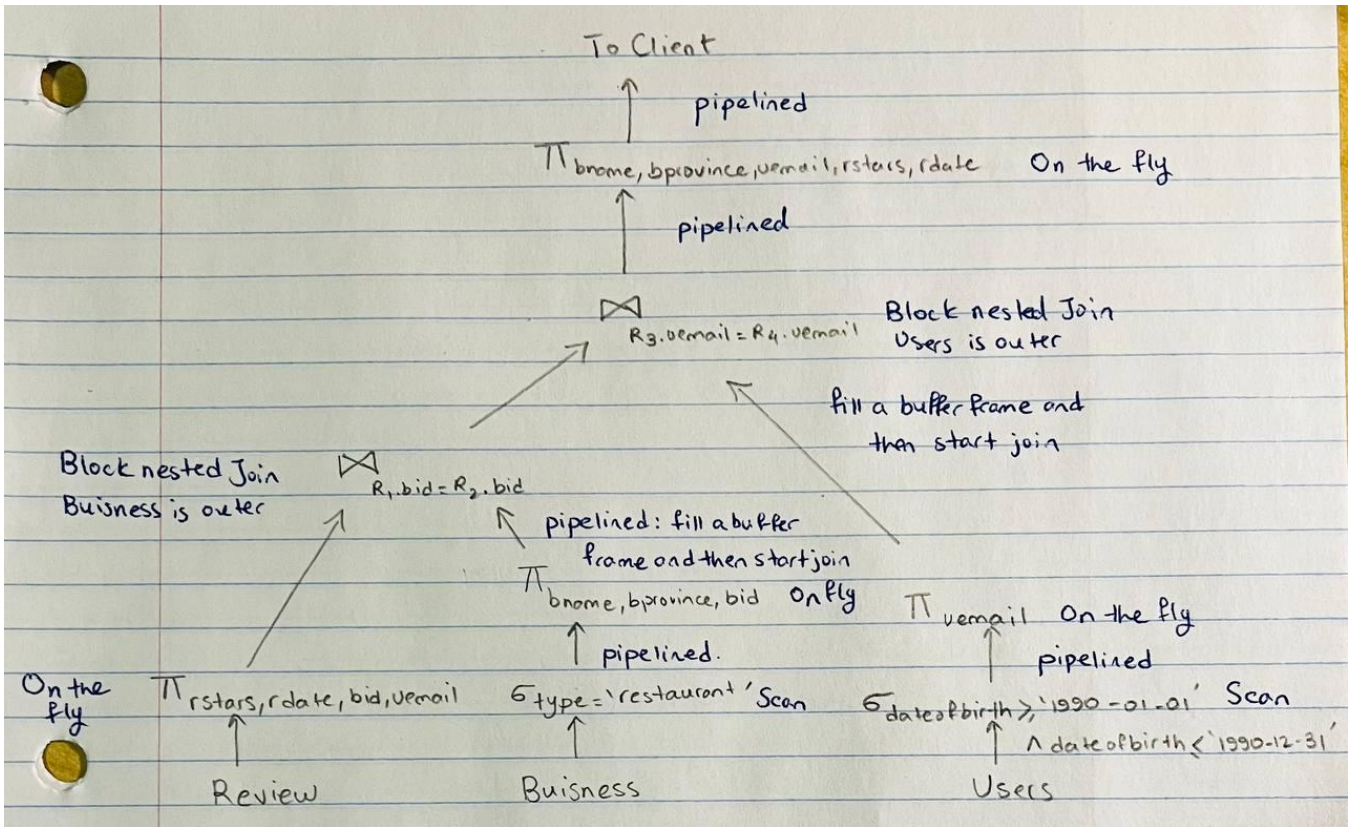


Figure 1: Execution plan tree

Ex. 4

The execution plan tree can be seen in Figure 2. We first performed algebraic optimization and pushed down selection and projection operators to reduce the resultsets flowing through operators.

We are only selecting reviews done on and after 2021-12-01. Knowing that the DB contains 4 years of reviews (4×365), the reduction facctor on the rdate condition of Reviews is $\frac{31}{365 \times 4}$ (December has 31 days). So

$$\sigma_{rdate \geq '2021-12-01}(Reviews) = 0.0212$$

Therefore, $0.0212 \times 90000 \times 20 = 38220$ tuples in the Reviews relation qualify.

For finding the most optimal join algorithm, we will ignore simple and page nested loop joins since they are the most inneficient algorithms studies. Furthermore, we cannot perform an index nested loop join since no index exists in the context of this question. We will therefore focus on block nested loop join and sort merge join algorithms.

1. If we have reviews be the outer table and pipeline qualifying tuples until a block is full for joining, we find the cost as follows (Note: We assumed a few extra memory buffers than 100).
cost = $ReviewPages + \lceil \frac{ReviewPages * 0.212}{100} \rceil \times UserPages = 275400 + \lceil 275400 \times 0.0212 \times 0.01 \rceil \times 2460 = 420540$ IO

This is very inneficient and having user be the inner page will not improve the IO cost (since we won't be able to pipeline the Reviews selection into the join).

2. If we apply sort merge join, we will need the following number of passes:
For Users: $1 + \lceil log_{100} \lceil {}^{2460}/_{100} \rceil \rceil = 2$ For Reviews: $1 + \lceil log_{100} \lceil {}^{275400}/_{100} \rceil \rceil = 3$
The number of passes are too large for Reviews. We can also ignore this option.

3. A better approach would be to perform selection and projection on both Reviews and Users before performing the join.
we found 38220 qualifying tuples in reviews projected on bid, rstars, and uemail (len = 10 + 1 + 30 = 41). Thus the total number of required pages will be $\frac{38220 \times 41}{4000} = 392 pages$. This does not fit into memory so we have to write it into disk. cost = 275400 pages reads + 392 pages write.

By the same logic, performing projection on users results in tuples of size lenght of uemail and uprovince = 30 + 2 = 32. With 90000 users, we will need $\frac{32 \times 90000}{4000} = 720 pages$. This does not fit into memory so we have to write it into disk. cost = 2460 pages reads + 720 pages write.

We can now compare the performance of Block nested loop join and sort merge join.
For block nested loop join:
if outer = users: cost = $720 + {}^{720}/_{100} \times 392 = 3543 IO$
if outer = reviews: cost = $392 + {}^{392}/_{100} \times 720 = 3215 IO$

Using sort merge join: We need 2 passes for each relation. in minimum, we need $3 \times 392 + 3 \times 720 = 3336 IO$

So we choose the block nested loop join with outer = reviews.
The join output will be pipelined into the sorting for the `GROUP BY` clause.
After the join (since we also did the projections before), the output will have tuples of size len(bid, rstar, uemail, uprovince) = 10 + 1 + 30 + 2 = 43 bytes. They will take up $\frac{38220 \times 43}{4000} = 411$ pages bytes.
Sorting will required 2 passes since: $1 + \lceil log_{100} \lceil {}^{411}/_{100} \rceil \rceil = 2$
Pass 0 can be read from the output of the Join but we will have to write the output to disk to compute aggregates. So the cost for the sort will be $3 \times 411 = 1233$ IO.

20000 buisnesses and 20 provinces exist. So 20000×20 groups can exist. Since 38220 tuples are output from the join, it is safe to assume that each will be in their own group. We need to scan 411 pages to compute aggregates. So 411 IO.
The output of these aggregates can be pipelined into the next sort. which will required 2 passes. Its input and output are both pipelined so the only IO are the pass 0 write and the pass 1 read. So IO = $2 \times 411 = 822$

Comparing the number of records to aggregates, it will be more efficient to not sort for finding distinct emails.
The total cost for the query will be: 2460 + 720 + 275400 + 392 + 3215 + 1233 + 822 = 284242 IO.

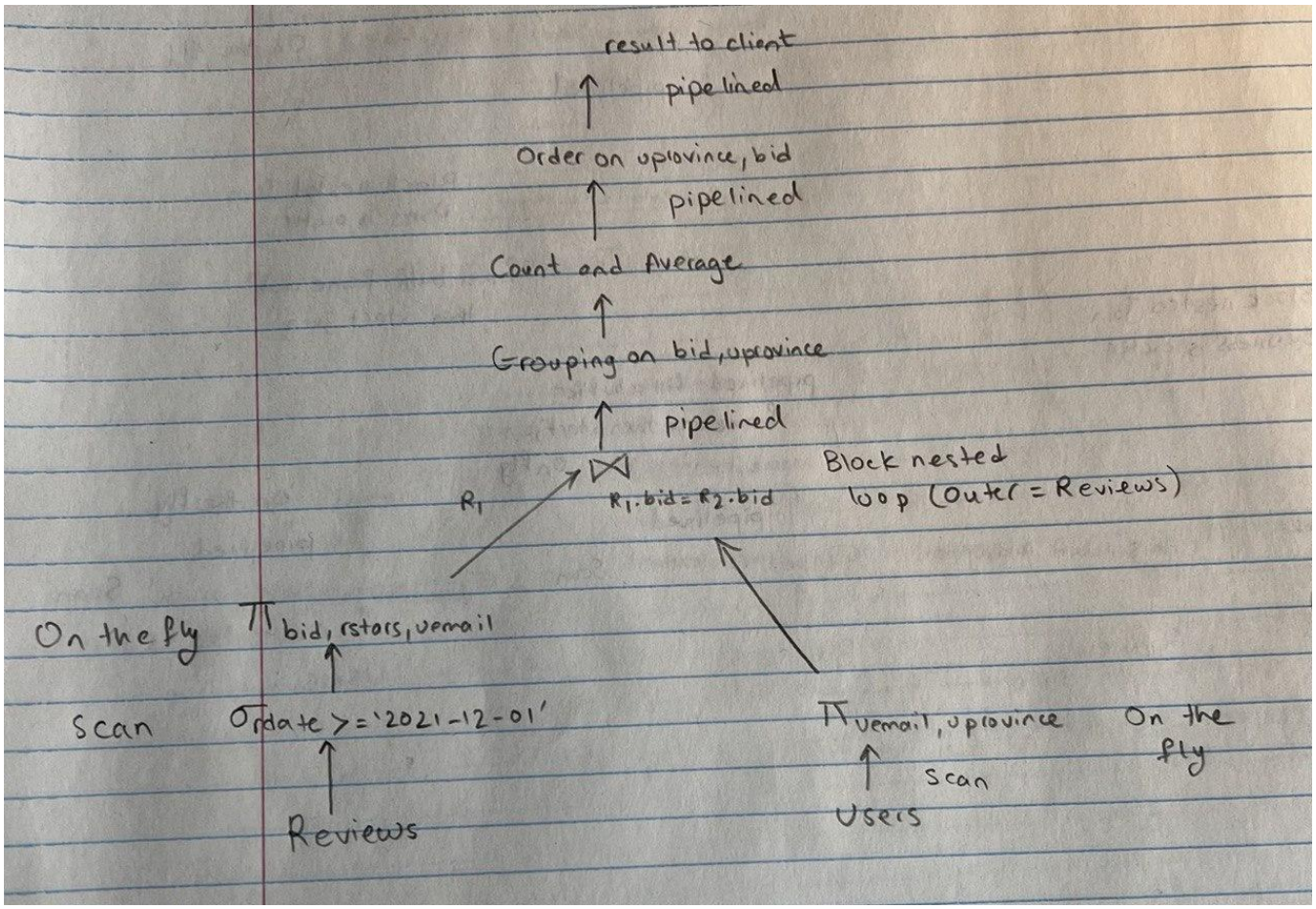


Figure 2: Execution plan tree